

第二章

高级语言及其语法描述

第二章 高级语言及其语法描述

- 2.1 程序语言的定义
- 2.2 高级语言的一般特性
- 2.3 程序语言的语法描述

2.1 程序语言的定义

- 任何语言实现的基础是语言定义。
- 程序语言由两方面定义：
 - 语法
 - 语义

2.1.1 语法

- 程序本质上是一定字符集(字母表)上的字符串。
- 语法：一组规则，用它可以形成和产生一个合式(well-formed)的程序。

语 法

- 字符串: $0.5 * X1 + C$
 - 单词符号: $0.5 \ X1 \ C \ * \ +$
 - 语法单位: $0.5 * X1 + C$

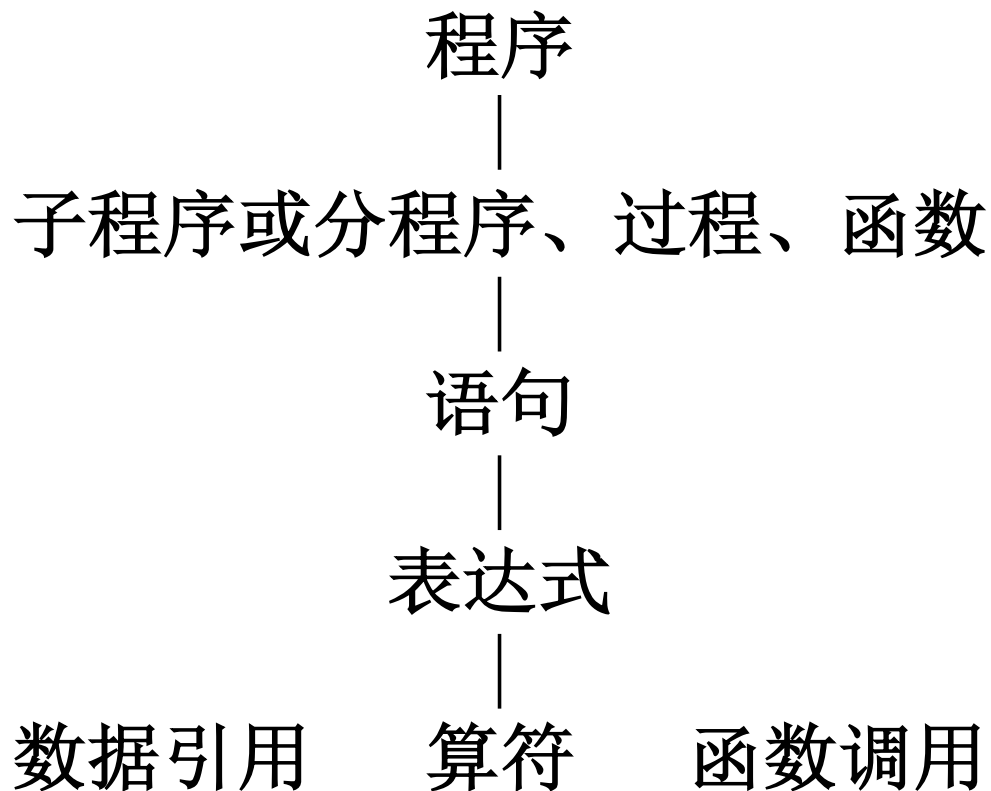
语 法

- 词法规则：单词符号的形成规则。
 - 词法规则规定了字母表中哪样的字符串是一个单词符号。
 - 单词符号是语言中具有独立意义的最基本结构。一般包括：常数、标识符、基本字、算符、界符等。
 - 描述工具：有限自动机

语 法

- 语法规则：语法单位的形成规则。
 - 如何从单词符号形成更大的结构（即语法单位）
 - 语法单位通常包括：表达式、语句、分程序、过程、函数、程序等
 - 描述工具：上下文无关文法

程序的层次结构



- 语法规则和词法规则定义了程序的形式结构，是判断输入字符串是否构成一个形式上正确（即合式）程序的依据。
- 定义语法单位的意义属于语义问题。

2.1.2 语义

- 语义：定义单词符号和语法单位的意义。
- 基于属性文法的语法制导翻译方法

2.3 程序语言的语法描述

- 语法结构的形式描述
 - 上下文无关文法
 - 语法分析树
 - 文法的二义性

2.3 程序语言的语法描述

- 1. 有穷字母表 Σ
 - 例如, $\Sigma = \{ a, b, c \}$
 $\Sigma' = \{ 0, 1 \}$
- 2. 符号
 - 字母表的每个元素称为一个符号。
 - 例如: a 、 b 、 c 是字母表 Σ 中的符号;
 0 、 1 是字母表 Σ' 中的符号。

2.3 程序语言的语法描述

3. 符号串/字

Σ 中的符号所构成的一个有穷序列。

例如，设有字母表 $\Sigma=\{ a, b, c \}$

则有符号串 $a, b, ab, ba, cba,$
 $abc...$

2.3 程序语言的语法描述

- 空字 ϵ

不包含任何符号的序列。

- 用 Σ^* 表示 Σ 上的所有符号串的全体, 空字 ϵ 也包括在其中。

例如: 设 $\Sigma = \{a, b\}$, 则

$$\Sigma^* = \{ \epsilon, a, b, aa, ab, ba, bb, aaa, \dots \}$$

- ϕ

不含任何元素的空集 $\{\}$ 。

符号串的运算

1. 符号串的连接

设 x 和 y 是符号串，则串 xy 称为它们的连接。

例如，设 $x=abc$ ， $y=10a$

则 $xy=abc10a$ ， $yx=10aabc$ 。

注意：对任意一个符号串 x ，
有 $\varepsilon x = x\varepsilon = x$ 。

- 集合的（连接）积

Σ^* 的子集U和V的（连接）积定义为：

$$UV = \{ \alpha\beta \mid \alpha \in U \ \& \ \beta \in V \}$$

即集合UV中的符号串是由U和V的符号串连接而成的。

例如：设 $A = \{ a, b \}$, $B = \{ c, d \}$

则 $AB = \{ ac, ad, bc, bd \}$

一般而言， $UV \neq VU$

符号串的运算

3. 符号串的幂运算-自身的n次（连接）积

设x是符号串, 则x的幂运算定义为:

$$x^0 = \varepsilon$$

注意: $x^0 \neq 1$

$$x^1 = x$$

$$x^2 = xx$$

$$x^3 = xxx$$

$$\begin{array}{c} \dots \\ x^n = \underbrace{xx \dots x}_n = x x^{n-1} (n > 0) \end{array}$$

符号串的运算

例如, 设 $x = abc$

则

$$x^0 = \varepsilon$$

$$x^1 = abc$$

$$x^2 = xx = abcabc$$

...

符号串的运算

4. 集合的幂运算-集合自身的n次（连接）积

设V是符号串的集合，则集合V的幂运算定义为：

$$V^0 = \{\varepsilon\}$$

$$V^1 = V$$

$$V^2 = VV$$

$$V^n = \underbrace{VV \dots V}_n = VV^{n-1} (n > 0)$$

符号串的运算

例如，设 $V = \{ a, b \}$ ，则

$$V^0 = \{ \varepsilon \}$$

$$V^1 = V = \{ a, b \}$$

$$V^2 = VV = \{ aa, ab, ba, bb \}$$

$$V^3 = VVV = V^2V$$

$$= \{ aaa, aab, aba, abb, baa, bab, bba, bbb \}$$

...

符号串的运算

5. 集合 V 的正则闭包 V^+ 与闭包 V^*

设 V 是符号串的集合，则

V 的闭包 V^* 定义为

$$V^* = V^0 \cup V^1 \cup V^2 \cup \dots \cup V^n \dots$$

闭包 V^* 中的每个符号串都是由 V 中的符号串经有限次连接而成。

V 的正则闭包 V^+ 定义为：

$$V^+ = VV^* = V^1 \cup V^2 \cup \dots \cup V^n \dots$$

符号串的运算

例如，设 $V = \{ a, b \}$ ，则：

$$V^* = \{ \varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots \}$$

$$V^+ = \{ a, b, aa, ab, ba, bb, aaa, aab, \dots \}$$

集合 V 的正则闭包表示 V 上符号串 a, b 构成的所有符号串的集合，

集合 V 的闭包比集合 V 的正则闭包多含一个空符号串 ε 。

2.3.1 上下文无关文法

- 文法：描述语言的语法结构的形式规则。
- 上下文无关文法：它所定义的语法范畴（或语法单位）是完全独立于这种范畴可能出现的环境的。

2.3.1 上下文无关文法

语法规则：

→：由…组成或定义为…

- (1) 〈句子〉 → 〈主语〉〈谓语〉〈间接宾语〉〈直接宾语〉
- (2) 〈主语〉 → 〈代词〉
- (3) 〈谓语〉 → 〈动词〉
- (4) 〈间接宾语〉 → 〈代词〉
- (5) 〈直接宾语〉 → 〈冠词〉 〈名词〉
- (6) 〈代词〉 → He
- (7) 〈代词〉 → me
- (8) 〈名词〉 → book
- (9) 〈冠词〉 → a
- (10) 〈动词〉 → gave

有了这些规则后，可以推导或产生出句子：

He gave me a book

从<句子>出发，反复把上述规则中“ $\rightarrow$ ”左边的成分替换成右边的成分。

语法规则:

- (1) <句子> → <主语><谓语><间接宾语><直接宾语>
- (2) <主语> → <代词>
- (3) <谓语> → <动词>
- (4) <间接宾语> → <代词>
- (5) <直接宾语> → <冠词> <名词>
- (6) <代词> → He
- (7) <代词> → me
- (8) <名词> → book
- (9) <冠词> → a
- (10) <动词> → gave

推导:

<句子>

⇒<主语><谓语><间接宾语><直接宾语>

⇒<代词><谓语><间接宾语><直接宾语>

⇒He <谓语><间接宾语><直接宾语>

⇒He <动词><间接宾语><直接宾语>

⇒He gave <间接宾语><直接宾语>

⇒He gave <代词><直接宾语>

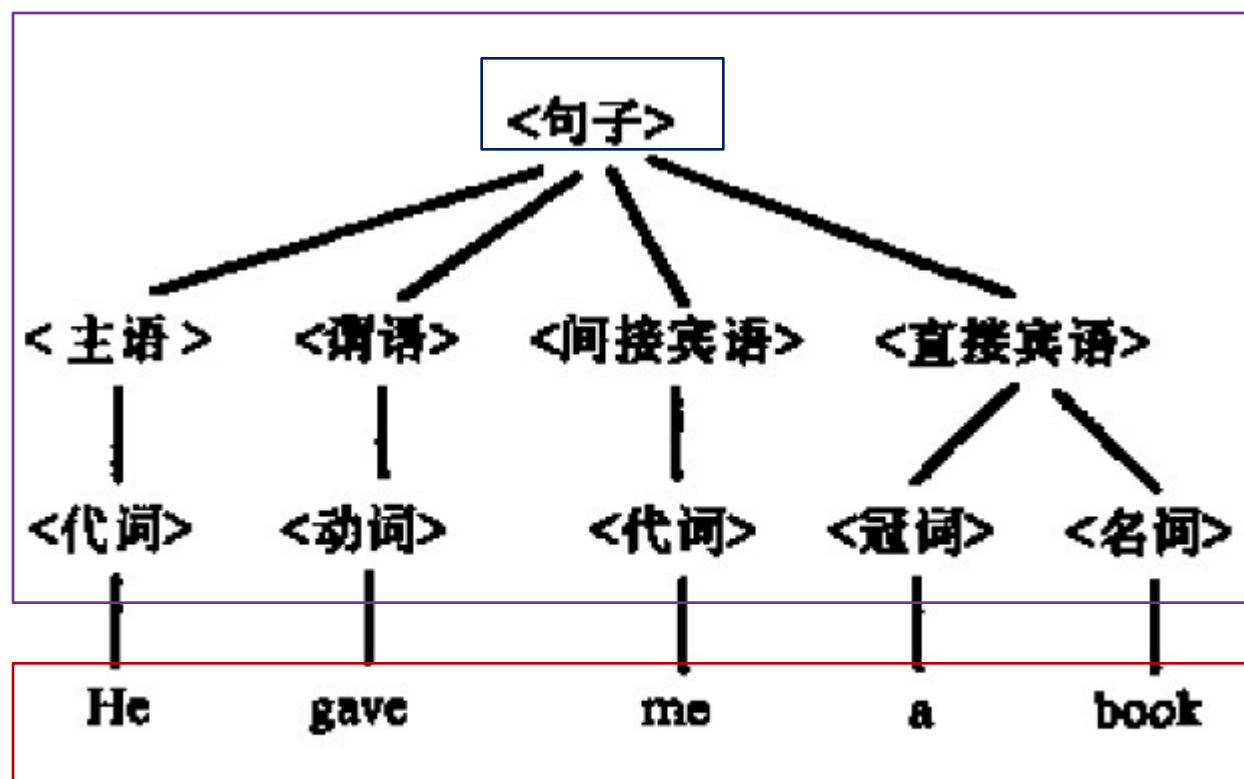
⇒He gave me <直接宾语>

⇒He gave me <冠词><名词>

⇒He gave me a <名词>

⇒He gave me a book

- 语法分析树



开始符号

非终结符号

终结符号

图 2.1 语法树

- 一个上下文无关文法G包括四个组成部分：一组终结符，一组非终结符，一个开始符号，以及一组产生式。
 - 终结符号：组成语言的基本符号，即单词符号，如基本字、标识符、常数、算符和界符等；
 - 非终结符号（语法变量）：代表语法范畴，如算术表达式、赋值句和分程序等；
 - 开始符号：特殊的非终结符，最终感兴趣的语法范畴；
 - 产生式：定义语法范畴的书写规则，形式是 $A \rightarrow \alpha$ 。

- 产生式: $A \rightarrow \alpha$

- A : 非终结符

- α : 由终结符或/与非终结符组成的符号串

例: 假定要定义一类含 $+$ 、 $*$ 的算术表达式, 定义可以这样给出:

变量是一个算术表达式;

若 E_1 和 E_2 是算术表达式, 则 $E_1 + E_2$ 、 $E_1 * E_2$ 、 (E_1) 也是算术表达式。

产生式: $E \rightarrow i$

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow (E)$$

- 上下文无关文法的定义:

一个上下文无关文法G是一个四元式

$G(V_T, V_N, S, P)$, 其中

- V_T : 非空有限集, 每个元素称为终结符号;
- V_N : 非空有限集, 每个元素称为非终结符号,
且 $V_T \cap V_N = \emptyset$;
- S : 开始符号, 是一个非终结符号;
- P : 产生式集合(有限), 每个产生式的形式是
 $A \rightarrow \alpha, \quad A \in V_N, \quad \alpha \in (V_T \cup V_N)^*$
- 开始符号S至少必须在某个产生式的左部出现一次。

- 例：定义只含 $+$, $*$ 的算术表达式的文法。

定义可以这样给出：

变量 i 是一个算术表达式；

若 E_1 和 E_2 是算术表达式，则 $E_1 + E_2$ 、 $E_1 * E_2$ 和 (E_1) 也是算术表达式。

文法定义：

$G(\{i, +, *, (,)\}, \{E\}, E, P)$ ，其中， P 由下列产生式组成：

$E \rightarrow i$

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

- 几点规定:

- (1)

$$P \rightarrow \alpha_1$$

$$P \rightarrow \alpha_2$$

...

$$P \rightarrow \alpha_n$$

可缩写为 $P \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$

其中, “|” 读成 “或”, α_i 称为P的一个候选式。

- (2)

为简便起见, 仅列出产生式和指出开始符号。

如上例, 可表示为:

$$G(E): E \rightarrow i | E + E | E * E | (E)$$

– (3)

大写字母A、B、C...或汉语词组代表非终结符号

小写字母a、b、c...代表终结符号

α 、 β 、 γ 等代表终结符和非终结符组成的符号串

- 文法与语言

从文法的开始符号出发，反复连续使用产生式，对非终结符施行替换和展开。

- 推导

如：文法 $G(E)$: $E \rightarrow i \mid E+E \mid E^*E \mid (E)$

$E \Rightarrow i$

$E \Rightarrow (E) \Rightarrow (i)$

$E \Rightarrow E+E \Rightarrow i+E \Rightarrow i+i$

$E \Rightarrow E^*E \Rightarrow E+E^*E \Rightarrow i+E^*E \Rightarrow i+i^*E \Rightarrow i+i^*i$

$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E^*E+E) \Rightarrow (i^*E+E) \Rightarrow (i^*i+E)$
 $\Rightarrow (i^*i+i)$

.....

推导每前进一步总是引用一条产生式，符号 $\Rightarrow$ 指仅
推导一步。

- 定义：称 $\alpha A \beta$ 直接推出 $\alpha \gamma \beta$ ，即

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

仅当 $A \rightarrow \gamma$ 是一个产生式，且 $\alpha, \beta \in (V_T \cup V_N)^*$ 。

- 如果 $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ ，则称这个序列是从 α_1 至 α_n 的一个**推导**。
- 若存在一个从 α_1 至 α_n 的推导，则称 α_1 可推导出 α_n 。

- $\alpha_1 \overset{+}{\Rightarrow} \alpha_n$: 从 α_1 出发, 经一步或若干步, 可推导出 α_n 。
- $\alpha_1 \overset{*}{\Rightarrow} \alpha_n$: 从 α_1 出发, 经0步或若干步, 可推导出 α_n 。

即 $\alpha \overset{*}{\Rightarrow} \beta$ 意味着: $\alpha = \beta$ 或者 $\alpha \overset{+}{\Rightarrow} \beta$ 。

- 假定 G 是一个文法, S 是它的开始符号, 如果 $S \overset{*}{\Rightarrow} \alpha$, 则称 α 是一个句型。

如：文法 $G(E)$: $E \rightarrow i \mid E+E \mid E^*E \mid (E)$

从开始符号 E 到 $(i*i+i)$ 的推导

最左推导

(1): $E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E^*E+E) \Rightarrow (i^*E+E)$
 $\Rightarrow (i^*i+E) \Rightarrow (i^*i+i)$

E 、 (E) 、 $(E+E)$ 、 (E^*E+E) 、 (i^*E+E) 、 (i^*i+E) 、 (i^*i+i)
是文法的句型；

最右推导

(2): $E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E+i) \Rightarrow (E^*E+i) \Rightarrow (E^*i+i)$
 $\Rightarrow (i^*i+i)$

E 、 (E) 、 $(E+E)$ 、 $(E+i)$ 、 (E^*E+i) 、 (E^*i+i) 、 (i^*i+i)
是文法的句型；

(3):

$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E^*E+E) \Rightarrow (E^*i+E) \Rightarrow (E^*i+i) \Rightarrow (i^*i+i)$

E 、 (E) 、 $(E+E)$ 、 (E^*E+E) 、 (E^*i+E) 、 (E^*i+i) 、 (i^*i+i)
是文法的句型；

从一个句型到另一个句型的推导过程**不是唯一的**。

- **最左推导**：任何一步 $\alpha \Rightarrow \beta$ 都是对 α 中的最左非终结符进行替换。
- **最右推导**：任何一步 $\alpha \Rightarrow \beta$ 都是对 α 中的最右非终结符进行替换。

如：文法 $G(E)$: $E \rightarrow i \mid E+E \mid E^*E \mid (E)$

从开始符号 E 到 $(i*i+i)$ 的推导

(1): $E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E^*E+E) \Rightarrow (i^*E+E)$
 $\Rightarrow (i*i+E) \Rightarrow (i*i+i)$

E 、 (E) 、 $(E+E)$ 、 (E^*E+E) 、 (i^*E+E) 、 $(i*i+E)$ 、 $(i*i+i)$ 是文法的句型；

定义：

- **句子**：仅含终结符号的句型是一个句子。
- **语言**：文法 G 所产生的句子的全体是一个语言，将它记为 $L(G)$ 。

$$L(G) = \{ \alpha \mid S \xRightarrow{+} \alpha \ \& \ \alpha \in V_T^* \}$$

- 例: $(i*i+i)$ 是文法

$G(E): E \rightarrow i \mid E+E \mid E^*E \mid (E)$

的一个句子。

证明:

$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E^*E+E) \Rightarrow (i^*E+E) \Rightarrow$
 $(i^*i+E) \Rightarrow (i^*i+i)$

$E, (E), (E^*E+E), \dots, (i^*i+i)$ 是句型。

□ 例2.1: 文法 $G_1(S)$:

$$S \rightarrow bA$$

$$A \rightarrow aA|a$$

$G_1(S)$ 的语言?

以b开头, 后跟一个或任意多个a的符号串

$$L(G_1) = \{ba^n | n \geq 1\}$$

- **递归**: 产生式左部的符号直接在右部出现。
- 用递归方式定义语法范畴是一种非常重要的手段。

- 例2.2: 文法 $G_2(S)$:
 $S \rightarrow AB$
 $A \rightarrow aA|a$
 $B \rightarrow bB|b$

$G_2(S)$ 的语言?

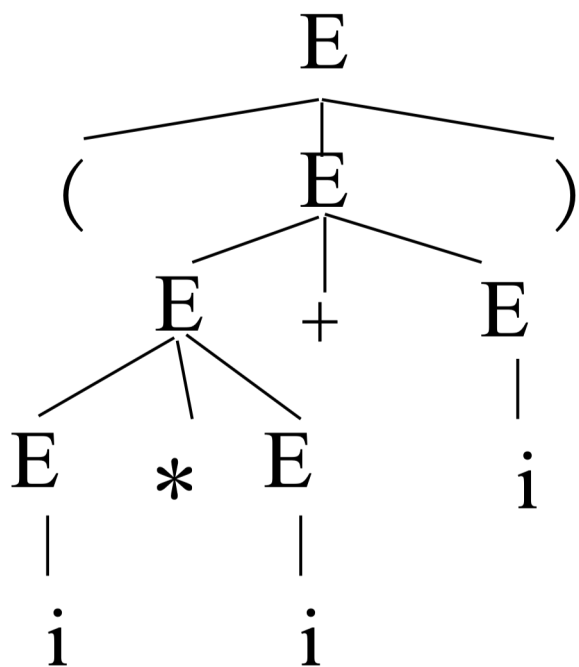
$$L(G_2) = \{a^m b^n | m, n \geq 1\}$$

2.3.2 语法分析树与二义性

- 用一张图表示一个句型的推导, 称为语法分析树, 或简称为语法树。

$G(E): E \rightarrow i \mid E+E \mid E * E \mid (E)$

$(i*i+i)$ 的语法树

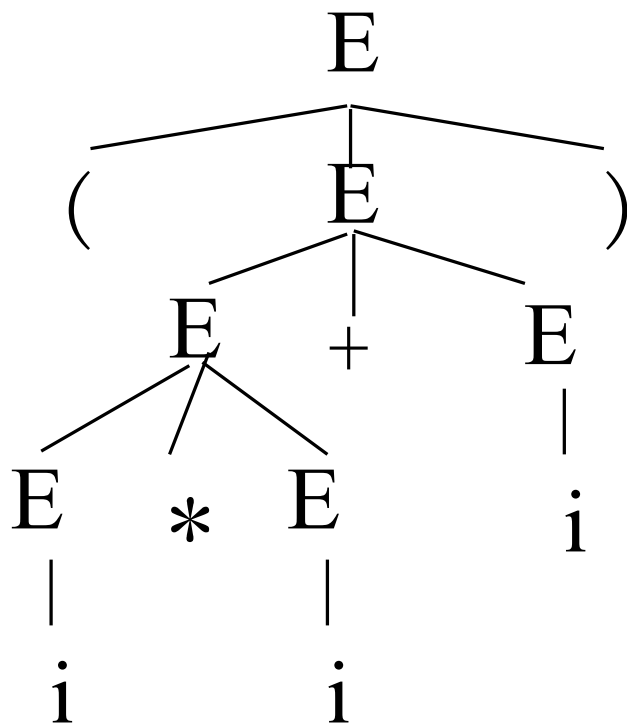


- 根结: 开始符号;
- 新结: 当某个非终结符被它的某个候选式所替换时, 候选式中自左至右的每个符号对应一个新结。
- 在一棵语法树生成过程中的任何时刻, 所有那些没有后代的端末结自左至右排列起来就是一个句型。

- 一棵语法树表示了一个句型种种可能的（但未必是所有的）不同推导过程，包括最左（最右）推导。
- 一棵语法树是不同推导过程的共性抽象。
- 坚持使用最左（最右）推导，一棵语法树等价于一个最左（最右）推导。
- 一个句型是否只对应唯一一棵语法树？它是否只有唯一的一个最左（最右）推导呢？

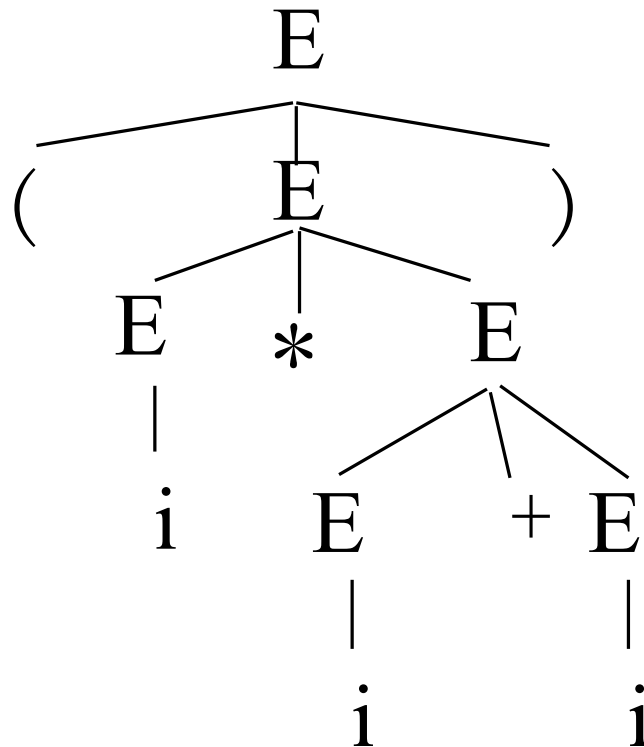
$G(E): E \rightarrow i \mid E+E \mid E^*E \mid (E)$

$E \Rightarrow (E) \Rightarrow (E+E)$
 $\Rightarrow (E^*E+E) \Rightarrow (i^*E+E)$
 $\Rightarrow (i^*i+E) \Rightarrow (i^*i+i)$



$(9*5+2): 47$

$E \Rightarrow (E) \Rightarrow (E^*E)$
 $\Rightarrow (i^*E) \Rightarrow (i^*E+E)$
 $\Rightarrow (i^*i+E) \Rightarrow (i^*i+i)$



$(9*5+2): 63$

- 如果一个文法存在某个句子对应两棵不同的语法树，则称这个文法是二义的。
- 若一个文法中存在某个句子，它有两个不同的最左（最右）推导，则这个文法是二义的。

$G(E): E \rightarrow i | E + E | E * E | (E)$ 是二义文法。

- 二义性问题是不可判定的，即不存在一个算法，它能在有限步骤内，确切地判定一个文法是否为二义的。
- 无二义文法：

$G(E):$

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow (E) \mid i$$

无二义文法:

$G(E)$:

$E \rightarrow T \mid E+T$

$T \rightarrow F \mid T * F$

$F \rightarrow (E) \mid i$

考虑句子 $(i*i+i)$

$E \Rightarrow T$

$\Rightarrow F$

$\Rightarrow (E)$

$\Rightarrow (E+T)$

$\Rightarrow (T+T)$

$\Rightarrow (T * F + T)$

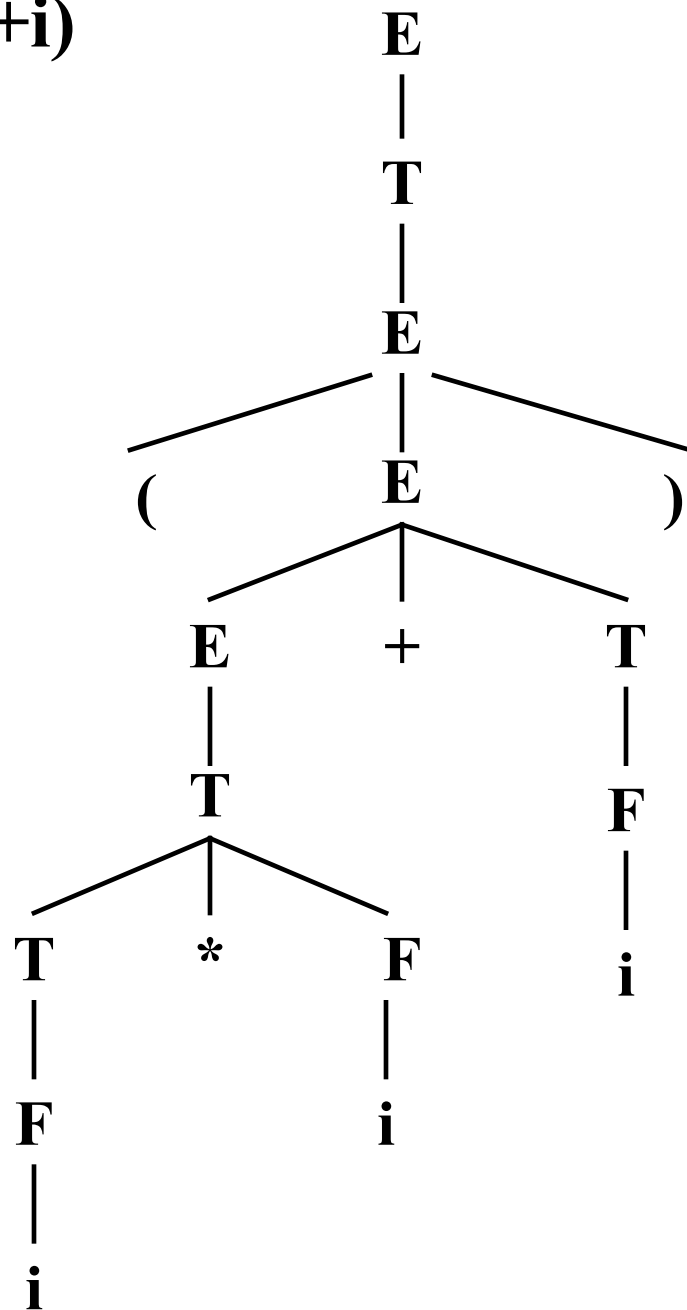
$\Rightarrow (F * F + T)$

$\Rightarrow (i * F + T)$

$\Rightarrow (i * i + T)$

$\Rightarrow (i * i + F)$

$\Rightarrow (i * i + i)$



无二义文法:

$G(E)$:

$E \rightarrow T \mid E+T$

$T \rightarrow F \mid T * F$

$F \rightarrow (E) \mid i$

考虑句子 $(i*i+i)$

$E \Rightarrow T$

$\Rightarrow F$

$\Rightarrow (E)$

$\Rightarrow (E+T)$

$\Rightarrow (T+T)$

$\Rightarrow (T * F + T)$

$\Rightarrow (F * F + T)$

$\Rightarrow (i * F + T)$

$\Rightarrow (i * i + T)$

$\Rightarrow (i * i + F)$

$\Rightarrow (i * i + i)$

